FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

CSE3433 SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

PROJECT TITLE:
GROUP 14
RECRUITX - EVENT-DRIVEN JOB RECRUITMENT & APPLICATION PROCESSING SYSTEM

INDIVIDUAL LOGBOOK

PREPARED FOR:
DR. MOHAMAD NOR BIN HASSAN

PREPARED BY:

NAME	MATRIC NO.
AINAA NADHIRAH BINTI ASMADI	S75246

TABLE OF CONTENT

1. Team Information:

Project Title: RecruitX - Event-Driven Job Recruitment & Application Processing System

Group Number: 14

Team members:

Name	Role
Nur Alin Hayani binti Ahmad Syukri	System Architect & Designer
Ainaa Nadhirah binti Asmadi	Backend & Integration Lead
Maryam Musfirah binti Ariff Hazizan	Requirements Analyst

2. Weekly Progress:

2.1 Week 4

Item	Description
Week	Week 4 (5 April - 9 April 2026)
Weekly Activities	- Consolidating all materials for the Project Proposal. - Documented the EDA style justification, problem description, objectives and role assignments.
Individual Contribution	- Authored the architectural justification portion of the proposal - detailing why an event broker/dispatcher approach solves the performance issues.
Challenges	Standardizing formatting styles across sections written by different team members to ensure a cohesive document layout.
Action Plans	Review the proposal draft against the Deliverable 1 checklist provided in the guidelines before submission.

2.2 Week 5

Item	Description
Week	Week 5 (12 April - 16 April 2026)
Weekly Activities	- Finalized, polished, and successfully submitted Deliverable 1: Project Proposal. - Initialized the team GitHub repository for version control.
Individual Contribution	= Created the project repository on GitHub, configured project files for NetBeans IDE integration - Set up main/development branch protection rules.

Item	Description
Challenges	Coordinating matching project configuration settings inside NetBeans among team members to prevent metadata conflicts.
Action Plans	- Transition to Phase 2 (Architecture Design) - Begin defining system components and boundaries.

2.3 Week 6

Item	Description
Week	Week 6 (19 April - 23 April 2026)
Weekly Activities	- Proceeded with Phase 2: Architecture Design - Focusing on System Decomposition. - Decomposed the system into 4 services: Application, Screening, Interview and Notification.
Individual Contribution	- Drafted the component decomposition table. - Specifically defined boundary statements ensuring that the Notification Service only consumes events.
Challenges	- Resisting the urge to allow direct database queries across services. -Had to strictly enforce that one service does not access another's database layer.
Action Plans	Map out the asynchronous communication and interaction flows between these 4 services.

2.4 Week 7

Item	Description
Week	Week 7 (26 April - 30 April 2026)
Weekly Activities	- Developed data ownership rules and interaction maps.

Item	Description
	- Designed the four mandatory architecture diagrams using Draw.io. -Consolidated all items into Deliverable 2.
Individual Contribution	Designed the Interaction Diagram mapping the specific asynchronous event lifecycle chain: ApplicationSubmitted -> ResumeScreened -> Shortlisted -> InterviewScheduled -> Notified.
Challenges	Properly modeling an asynchronous event bus broker layout within a sequence diagram format without making it look like a synchronous REST call.
Action Plans	Submit Deliverable 2 on time and transition into Phase 3 (Detailed Design)

2.5 Week 8

Item	Description
Week	Week 8 (10 May - 14 May 2026)
Weekly Activities	Modeled the precise Java object data models for the ApplicationSubmitted event data payload, utilizing strongly-typed properties and primitive data attributes instead of loose data strings.
Individual Contribution	- Modeled the precise Java object data models for the ApplicationSubmitted event payload - Ensuring lightweight reference attributes are used instead of large binary file blocks.
Challenges	Ensuring event classes are deeply decoupled and completely serializable across memory states without cross-referencing parent UI controller contexts.
Action Plans	Set up individual package modules inside the NetBeans project structure to begin writing standalone service logic.

2.6 Week 9

Item	Description
Week	Week 9 (17 May - 21 May 2026)
Weekly Activities	- Completed development environment configurations exclusively within NetBeans IDE. - Conducted internal class structure and codebase architecture alignment checks within the team.
Individual Contribution	Programmed the core asynchronous Event Broker Emulator module utilizing custom Java event listener patterns and multi-threaded queues to simulate a message broker inside NetBeans.
Challenges	Ensuring our localized Java-based event bus framework completely avoids any hidden internal synchronous execution blocking.
Action Plans	Begin the standalone implementation of the individual producer and consumer application logic packages.

2.7 Week 10

Item	Description
Week	Week 10 (24 May - 28 May 2026)
Weekly Activities	- Implementing Core Components. - Developed standalone Java modules for the Application Service (Producer) and the Screening Service (Consumer/Producer).
Individual Contribution	- Coded the backend processing logic within NetBeans for the Application Service to accept system requests - Write a temporary processing state, and publish an asynchronous message event.
Challenges	Debugging multi-threaded concurrency issues where the Screening Service tried to process an event before the Event Broker routing

Item	Description
	handlers had fully bound the reference.
Action Plans	Implement the remaining downstream consumer modules (Interview and Notification Services) using standard Java patterns.

2.8 Week 11

Item	Description
Week	Week 11 (31 May - 4 June 2026)
Weekly Activities	- Communication Implementation and End-to-End Flow Integration. -Consolidated the NetBeans project source directory
Individual Contribution	Integrated the backend Java event chain with the frontend user views, validating that submitting an application successfully triggers reactive thread execution across services.
Challenges	Managing edge-case execution paths where a corrupted string input could potentially halt the background screening worker threads.
Action Plans	Transition into Phase 5 (Evaluation, Code Refinement and Final Report Writing).

2.9 Week 12

Item	Description
Week	Week 12 (7 June - 11 June 2026)
Weekly Activities	- Conducted comprehensive internal testing and debugging on the core integrated packages. - Tested basic system edge cases and evaluated Quality Attributes against design targets.

Item	Description
Individual Contribution	- Refactored decoupled Java event handlers. - Explicitly verified that removing or disabling the Notification Service code did not break the recruitment intake execution loop.
Challenges	- Standardizing descriptive logging print streams output to the NetBeans console during async execution so that observers can track event propagation clearly.
Action Plans	Compile the final project report chapters and assemble individual reflection statements.

2.10 Week 13

Item	Description
Week	Week 13 (14 June - 18 June 2026)
Weekly Activities	- Completed Deliverable 4: Project Report & Individual Logbook. - Assembled the full comprehensive report.
Individual Contribution	- Wrote the "Architecture Evaluation" section of the document - Explaining how the decoupled event structure satisfies the target requirements for high scalability.
Challenges	Restructuring and condensing extensive code segments into clear, concise diagrams and code-snippet figures suitable for inclusion in the formal text.
Action Plans	Submit Deliverable 4 and coordinate with team members to run rehearsal runs of our system for the final demonstration.

3.0 GenAI Usage Record

Field	Details
Date (Week)	Week 4 (5 April - 9 April 2026)
Tool Used	Google Gemini
Purpose	Formulating the initial project problem statement and architectural core objectives.
Prompt Used	"Help me draft 3 clear architectural objectives for an event-driven system handling job recruitment workflows to prevent core thread blocking."
Output Summary	Provided 3 structured system objectives focusing on decoupled micro-components, async request paths, and thread optimization.
How It Was Used	Integrated directly into the problem understanding phase of our submitted Project Proposal document.

Field	Details
Date (Week)	Week 6 (19 April - 23 April 2026)
Tool Used	Google Gemini
Purpose	Organizing the structure of a component responsibility matrix table.
Prompt Used	"Generate a clean Markdown table framework containing columns for Service Name, Architectural Core Responsibility, and Explicit Exclusions."
Output Summary	Provided an empty Markdown table skeleton

Field	Details
	structure with columns organized as requested.
How It Was Used	Used as the foundation for organizing our Component Decomposition Table.

Field	Details
Date (Week)	Week 8 (10 May - 14 May 2026)
Tool Used	Google Gemini
Purpose	Designing structural decoupling strategies for distributed Java components.
Prompt Used	"What are best practices in pure Java architecture to design custom payload classes for asynchronous event dispatching without using JSON libraries?"
Output Summary	Recommended using strictly typed Plain Old Java Objects containing basic attributes and passing immutable parameter objects.
How It Was Used	Guided our internal package interfaces layout and structured our interface event schemas.

Field	Details
Date (Week)	Week 10 (24 May - 28 May 2026)
Tool Used	Google Gemini
Purpose	Resolving multi-threaded event listener binding latency issues in Java.

Field	Details
Prompt Used	"How do I prevent background thread worker concurrency race conditions in Java when publishing async events to active custom listeners?"
Output Summary	Suggested managing listener updates inside synchronized blocks or utilizing a ConcurrentLinkedQueue to sequentially decouple worker assignments.
How It Was Used	Used to optimize async processing stability within our NetBeans project event bus handler loop.

Field	Details
Date (Week)	Week 12 (7 June - 11 June 2026)
Tool Used	Google Gemini
Purpose	Structuring code quality evaluation metrics for non-functional testing verification.
Prompt Used	"How do I explicitly measure and document maintainability and component coupling metrics for an asynchronous Java event system during unit evaluation testing?"
Output Summary	Provided specific criteria frameworks targeting system stability when worker subsystems are removed, checking interface response paths.
How It Was Used	Used to form the testing methodology we followed when analyzing and refactoring code paths.